Experiment/~~Assignment~~ No 1

Title: Write a python program to manage the borrowing records of books in library. Implement the following functionalities:

1] Compute the average number of books borrowed by all library ~~memories~~ members.

2] find the book with highest and lowest number of borrowings in the library.

3] Count the number of members who have not borrowed any books (denoted by a borrow count of 0).

4] Display the most frequently borowed book (i.e the mode of borrow counts)

After performing, determine the time and space complexity of each operation.

Objectives:-

1. To provide hands-on experience with basics and advanced data structureds.
2. To apply Data structures for real-world problem sluing.

Learning objectives:

To understand importance of data structures in context of writing efficient programs.

Learning Outcomes:-

1. Analyze basic searching and sorting algorith to solve problems and evaluate thier effic in diffrent scenarios.

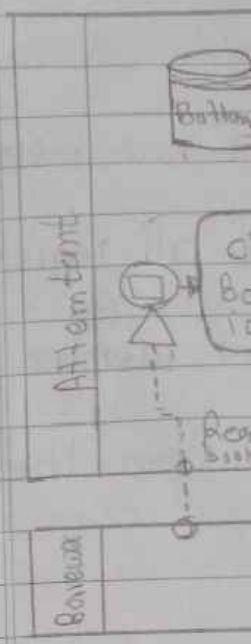
2. Understand and analyze various types of structures and algorithms.

Theory :-

A library Managment System using Python GU typically involves creating an application tha manages book records, member information an borrowing process. can build an interactive system that allow users to add, update a search for books efficiently.

The architecture of a library Managment Sys (LSM) using Python typically follows layered structures that enhances modularity and maint

Process model for books that commonly opera in libraries.



The com

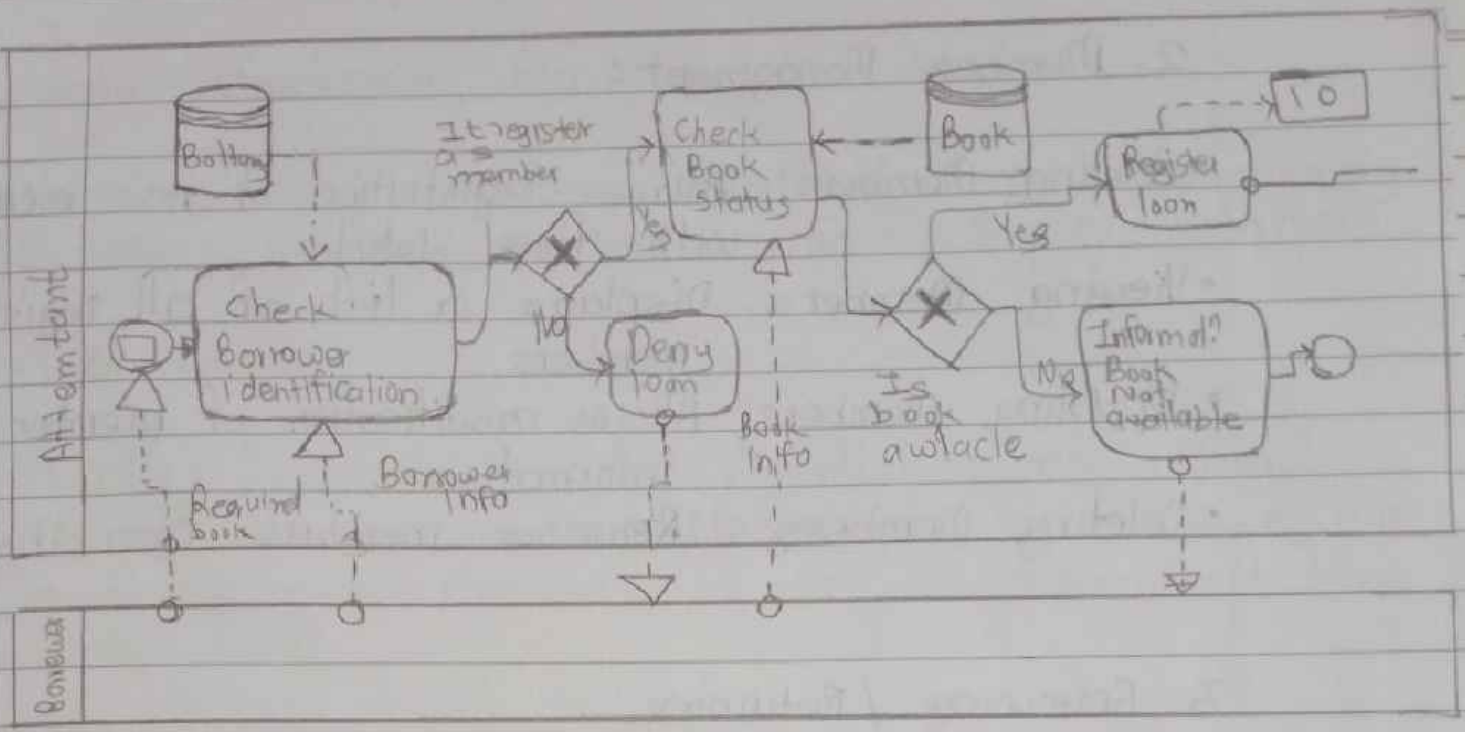
1. Book M

• Addition

• Viewing
library,

• Updating

• Removing



The component and functionalities

1. Book Managment:

- Addition Books: Allows librarians to add new books to the system, including details like title, author, ISBN and quantity.
- Viewing Books: Displays a list of all books in the library, including thier details.
- Updating Books: Enables modifications of book information (e.g., quantity, location).
- Removing Books: Allows for deleting books from the system.

2. Member Managment:

- Adding Members: Allows registration of new mem with thier details.
- Viewing Members: Displays a list of all regis members.
- Updating Members: Allows modification of member information
- Deleting Members : Removes members from the

3. Borrowing / Returning:

- Borrowing Books: Links a book to a member's l and Records the borrowing date and due to d
- Returning Books: Updates the book's availability and the borrowing record.
- Overdue Books: Tracks books with a past due date and may include a mechanism for fines.

Space complexity

- Defination: Space complexity measures the am of memory or storage space an algorithm r to complete its execution, relatives to the inpu size (n)
- Component: It considers both the fixed part (space for code, constants, simple variable and the variable part (space for dynamic ob

data stru

- Notation: I is also C

- Examples:

- O(1) (amount o swapping Bubble so

- O(n) (L groces input st structures

- O(n²) proportion seen in based

Time Co

- Definati of th as a

- focus: or C rather

data structures, recursion stack etc).

- Notation: Like ~~the~~ time complexity, space complexity is also commonly expressed using Big O notation.

- Examples:

  - O(1) (constant space): Algorithm using a fixed amount of memory regardless of input size, like swapping two numbers or an in-place sort like Bubble sort.

  - O(n) (Linear space): Algorithm where memory usage grows linearly with input size, such is creating ~~input size~~ an array of 'n' elements or using data structures like linked lists or stacks.

  - O(n^2) (Quadric Space): Algorithm requiring memory proportional to the square of input size, often seen in algorithm that create two-dimensional arrays based on the input size.

Time Complexity

- Defination: Time complexity measures the amount of time an algorithm takes to run to completion as a function of the input size (n).

- focus: It focuses on the number of operations or computational steps the algorithm performs, rather than the actual time taken, which can

be influenced by factors like the progra language, hardware, and operating system.

- Notation: The complexity is typically expres using asymptotic notation like Big O, Big (Ω) and Big Omega (Ω), with Big O (worst scenario) being the most commonly used.

- Examples:

- O(1) (constant time): Operations that betake th same amount of time regardless of input size, accessing an element in an array by its index.

- O (log n) (Logarithmic Time): Operations where the taken increases logarithmically with input size, binary search on a sorted array.

- O(n) (Linear Time): Operations where the time increases linearly with input size, like iterating through an array or linked list to find an ele

- O(n log n) (Log-Linear / Linearithmic Time): Comm efficient sorting algorithms like Merge Sort, where time grows proportionally to the input size mu by the logarithm of the input size.

- O(n²) (Quadratic Time): Operations involving neste where the time taken grows proportionally to the of the input size, selection sort, and Insertion

Conclusion:

By leveragi programing princi is efficient, us specific needs. of new tech evolve alongsi and informati

****

Time and

Let:

- n= numb
- m= total
- u= num

1) Average

- Time Co
- Space C

2) Highest

- Time Com
- Space C

Conclusion:

By ~~heavi~~ leveraging Python's capabilities and object-oriented programing pinciples. libraries can create a system that is efficient, user-friendly, and adaptable to thier specific needs. The potential for growth and integration of new tecnologies ensures that the system can evolve alongside the changing landscape of libraries and information mancagment.

********** Library Managment system complexity **********

Time and Space Complexity Analysis.

Let:

- n = number of members
- m = total number of borrowed books
- u = number of unique books

1) Average books borrowed per member.

- Time Complexity : O(n)
- Space Complexity: O(1) (ignores input storage)

2) Highest and lowest borrowed books

- Time Complexity : O(u)
- Space Complexity: O(1) (additional, beyond the Counter object)

3) Members with 0 borrowings

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

4) Most frequently borrowed book

- Time complexity: $O(u)$
- Space Complexity: $O(k)$ where k is the number of books with max frequency ($\leq u$)